

DAY 1 — PRACTICAL TASKS WORKBOOK

Session: 4 Hours | 12 Tasks | Individual & Pair Exercises

Complete each task using the AI tool of your choice. Write your prompts and observations in the spaces provided. Tasks alternate between general business scenarios and software development contexts.

Participant Name	Date
Maiyada Albarwani	September 9, 2026
Job Title	Group
	Batch 3

Time Block	Section	Tasks	Focus Area
0:00 - 0:40	A — AI Understanding	1-2	AI concepts, tools overview, responsible use
0:40 - 1:30	B — Prompt Engineering	3-5	CRTFC framework, prompt quality, advanced techniques
1:30 - 2:30	C — Communication & Docs	6-8	Emails, documentation, meeting summaries
2:30 - 3:20	D — Data & Code Tasks	9-10	Document analysis, code-related AI tasks
3:20 - 4:00	E — Workflow Design	11-12	Automation templates, personal AI workflow plan

Blue tasks = General business context | Purple tasks = Software development context | Mixed = Both perspectives combined

SECTION A | AI Concepts, Tools & Responsible Use · 0:00 - 0:40

Complete Tasks 1-2 in this section. Start from memory, then verify using your AI tool. The goal is to build a solid conceptual foundation before applying AI to real tasks.

Task 1 — AI Knowledge Audit & Tool Exploration 20 min · Beginner

- 1. Without using AI, write your own definition of AI and list what you think it can and cannot do.
- 2. Ask an AI tool to explain what a Large Language Model is in simple terms.
- 3. Compare the AI's explanation to yours. Note 1 concept that was new or surprising.
- 4. Try a second tool with the exact same prompt and note differences.
- 5. Ask for the top 5 use cases for AI tools in software development.

Deliverable: Completed tables + 2 tool comparison notes + developer use cases listed.

My Definition of AI (before AI tool)	AI's Explanation (from the tool)
AI is basically a computer program that can learn from data and do things that normally need a human brain, like answering questions, recognizing pictures, or writing text. It's not actually "thinking," it's just really good at spotting patterns.	A Large Language Model is a program trained on huge amounts of text so it can predict and generate human-like language — kind of like your phone's autocomplete, but way more advanced and able to hold a full conversation.
AI CAN (your view)	AI CANNOT / Risks (your view)
1. Answer questions quickly 2. Summarize long text 3. Help write and fix code 4. Translate between languages	1. Truly understand feelings 2. Guarantee its answers are correct 3. Keep company data private on its own 4. Replace human judgment completely

Difference I noticed between Tool 1 and Tool 2:

ChatGPT's explanation felt more casual and chatty, almost like a friend explaining it over coffee. Claude's answer was more structured, step by step, and a bit more precise with the wording. Both covered basically the same idea though — I didn't expect them to be that close.

Top 5 AI use cases for software development (from AI's answer):

- 1. Writing and debugging code faster
- 2. Generating unit tests automatically
- 3. Explaining unfamiliar or legacy code
- 4. Writing documentation and code comments
- 5. Reviewing code for bugs and security issues

Task 2 — Responsible AI: Risk Scenarios for Developers

20 min · Beginner

1. Read each scenario, mark Safe or Risky, and give a one-sentence reason.
2. For Risky ones, write a safer alternative.
3. Ask the AI for the top 5 data privacy risks and compare to your own assessment.

Deliverable: Completed risk table + alternative safe approaches written.

Scenario	Safe / Risky?	Reason / Safe Alternative
You paste a function from your company's payment processing module to get a code review	Risky	That code is sensitive/proprietary — better to strip out company names and identifiers, or rewrite it as a generic example before pasting it in.
You ask AI: "Explain how JWT token refresh logic works" without pasting any code	Safe	Nothing specific to our company is shared, it's just a general concept question.
You paste your production server's nginx config file including the server IP and domain names	Risky	Real IPs and domains could help someone target our servers — mask/replace them with placeholders first.
You describe a bug scenario: "I have a function that processes user objects — it crashes when email is null"	Safe	No real code or data included, just a plain description of the problem.
You upload a CSV of customer order history (names, emails, order amounts) to ask for analysis	Risky	That's real customer personal data — should anonymize names/emails or use dummy data instead.
You ask: "Write a Python script to parse CSV files with columns: order_id, amount, status"	Safe	Only column names are mentioned, no actual customer data is shared.
You paste your AWS .env file to ask AI why your connection string isn't working	Risky	That file has real credentials/secrets — if leaked, someone could access our AWS account. Redact the actual keys and just describe the error instead.
You ask AI to suggest database index strategies for a table with 10 million rows	Safe	This is a general best-practice question, no real data or schema shared.

Comparison with AI's answer on top 5 data privacy risks:

The AI listed things like: leaking credentials/API keys, sharing customer personal data, exposing proprietary source code, uploading regulated data (health/financial info), and not knowing exactly how the tool stores or reuses what you paste in. That lines up pretty closely with what I flagged as "Risky" above.

SECTION B | Prompt Engineering Practice · 0:40 - 1:30

Tasks 3-5 build your prompt writing skills from basic to advanced using the CRTFC framework (Context, Role, Task, Format, Constraints).

Task 3 — Spot and Fix Weak Prompts

15 min · Beginner

Deliverable: 3 improved prompts written + result quality ratings.

Weak Prompt 1 (General): "Write documentation"

Why is it weak? No context on what needs documenting, no format, way too vague.

CONTEXT	I have a Python function that processes customer orders and it needs documenting for the rest of my team.
ROLE	Act as a senior software engineer.
TASK	Write a clear docstring explaining what the function does, its parameters, and what it returns.
FORMAT	Google-style docstring, under 10 lines.
CONSTRAINTS	Don't change the actual code logic, just add the documentation.

Result quality: ☒ **Much better** ☐ Slightly better ☐ No improvement

Weak Prompt 2 (Dev): "Help me with my code"

Why is it weak? Doesn't say what the code does, what's broken, or what kind of help I actually need.

CONTEXT	I'm building a login feature in Node.js and it crashes when a user submits an empty password field.
ROLE	Act as a backend developer.
TASK	Find the bug causing the crash and suggest a fix.
FORMAT	Show the corrected code plus a short explanation of the fix.
CONSTRAINTS	Keep the existing function structure, don't rewrite the whole file.

Result quality: ☒ **Much better** ☐ Slightly better ☐ No improvement

Weak Prompt 3 (General): "Make a summary"

Why is it weak? No mention of what to summarize or how long the summary should be.

CONTEXT	I have a 3-page report from our sprint planning session.
ROLE	Act as a project coordinator.
TASK	Summarize the main points of the report.
FORMAT	5 bullet points, plain language.
CONSTRAINTS	Keep the whole thing under 100 words.

Result quality: ☐ Much better ☒ **Slightly better** ☐ No improvement

Weak Prompt 4 (Dev): "Fix the bug"

Why is it weak? No code was included and no description of what's actually going wrong.

CONTEXT	I have a function that calculates order totals and it's sometimes returning negative numbers.
ROLE	Act as a Python developer.
TASK	Identify the bug and fix it.
FORMAT	Return the fixed function with short comments explaining the change.
CONSTRAINTS	Don't change the function name or its parameters.

Result quality: ☒ **Much better** ☐ Slightly better ☐ No improvement

Weak Prompt 5 (General): "Write an email about the meeting"

Why is it weak? Doesn't say who it's going to, what the meeting covered, or what tone to use.

CONTEXT	We had a sprint planning meeting today and the launch date got pushed back by a week.
ROLE	Act as a project manager.

TASK	Write an email updating the client on the new timeline.
FORMAT	Professional email, under 150 words.
CONSTRAINTS	Keep it positive, don't get into internal blockers in detail.

Result quality: ☒ **Much better** ☐ Slightly better ☐ No improvement

Task 4 — Build a Level-4 Expert Prompt

20 min · Intermediate

Deliverable: Full CRTFC prompt + follow-up refinement prompt + quality rating.

I chose **Scenario: A** (General — steering committee status email)

CONTEXT	I'm a project manager at a mid-size software company wrapping up sprint 14, and leadership expects a weekly status update.
ROLE	Act as an experienced project manager.
TASK	Write a status email to the steering committee covering milestone progress, one key risk, and the goal for next sprint.
FORMAT	Professional email, under 200 words, short subject line.
CONSTRAINTS	Keep it readable for non-technical execs, don't get too into the weeds.

My Full Prompt:

"I'm a project manager wrapping up sprint 14 for our product launch. Act as an experienced project manager and write a status email to our steering committee. Cover our milestone progress this sprint, one key risk (the staging deployment pipeline being broken), and our goal for next sprint. Keep it under 200 words, professional tone, readable for non-technical execs, with a short subject line."

Follow-up refinement prompt:

"Can you make the risk section sound less alarming but still make it clear it needs attention, and add one short line about the revised launch date?"

Initial result quality: ☐ Excellent ☒ **Good** ☐ Needs work

After refinement: ☒ **Excellent** ☐ Good ☐ Needs work

Task 5 — Advanced Prompting Techniques

15 min · Advanced

Deliverable: 3 technique examples attempted + observations noted.

Technique A — Chain of Thought

Prompt: "Explain step by step how a JWT token is validated on an API server. Then identify 3 potential security vulnerabilities in a basic implementation."

Response summary: It walked through the steps — extract the token from the header, verify the signature against the secret/public key, check the expiry date, then check claims like issuer and audience. For vulnerabilities it flagged: not checking token expiration, using a weak or hardcoded secret key, and not validating the signing algorithm (which can lead to an algorithm-confusion attack).

Technique B — Few-Shot Prompting

"Here are 2 examples of the comment style I want:
Example 1: # Calculates total price including tax
Example 2: # Validates that email field is not empty
Now write a comment in the same short, one-line style for this function: `def get_user_by_id(user_id): return db.query(User).filter(User.id == user_id).first()`"

Technique C — Iterative Refinement (all 3 prompts)

1. "Write a Python function that fetches a user by ID from a database."
2. "Now add error handling for when the user isn't found or the database connection fails."
3. "Now add logging for both successful lookups and errors, and write one unit test for the 'user not found' case."

Task 6 — Professional Email Writing

20 min · Intermediate

Deliverable: 2 complete email prompts + both emails reviewed and rated.

Scenario 1: General — Launch Delay Approval

CONTEXT	I'm a product manager and QA just found critical bugs that mean we can't launch on schedule.
ROLE	Act as a product manager writing to senior leadership.
TASK	Write an email requesting approval for a 2-week launch delay due to QA findings.
FORMAT	Professional email, under 200 words, include a revised timeline.
CONSTRAINTS	Stay confident and solution-focused, don't sound like we're panicking.

Full prompt:

"Act as a product manager. Write an email to senior management requesting approval for a 2-week delay in our product launch due to critical QA findings. Keep it under 200 words, professional and confident (not panicked), and include a revised timeline moving the launch from March 10 to March 24."

☐ Email is professional & ready to send ☒ **Needs minor edits** ☐ Needs major revision

Scenario 2: Dev — Emergency Production Access

CONTEXT	I'm a backend developer and we have a P1 incident affecting 15% of users on our checkout service.
ROLE	Act as a backend developer.
TASK	Write an email to DevOps requesting emergency access to production logs.
FORMAT	Short, direct email, bullet points for the details.
CONSTRAINTS	Make the urgency obvious, include affected service, symptoms, and access needed.

Full prompt:

"Act as a backend developer. Write an email to the DevOps team requesting emergency access to production logs to investigate a P1 incident impacting 15% of users on our checkout service. Include: the affected service (checkout-api), symptoms (intermittent 500 errors and timeouts), and what I need access to (read access to production logs for the last 24 hours). Keep it short, direct, and clearly mark it as urgent."

☒ **Email is professional & ready to send** ☐ Needs minor edits ☐ Needs major revision

BONUS — Tone Variation:

Asked AI to make it "more urgent" — it added the word URGENT to the subject line, shortened the sentences, and moved the actual request for access up to the very first line instead of leaving it in the middle of the email.

Task 7 — Technical Documentation Generation

25 min · Intermediate

Deliverable: 3 documentation prompts written + one "least editing needed" observation.

```
def process_user_order(user_id, items, discount_code=None):
    total = sum(item['price'] * item['qty'] for item in items)
    if discount_code:
        discount = get_discount(discount_code)
        total = total * (1 - discount)
    if total < 0:
        raise ValueError('Total cannot be negative')
    return {'user_id': user_id, 'total': round(total, 2), 'items_count': len(items)}
```

Doc Type 1 — Docstring Prompt:

"Add a Google-style docstring to this Python function explaining what it does, its parameters, its return value, and any exceptions it raises."

Doc Type 2 — README Section Prompt:

"Write an 'Installation & Setup' section for a README.md file for a Node.js REST API. Include: prerequisites, how to clone the repo, install dependencies, configure the .env file, and run the dev server. Use numbered steps and code blocks."

Doc Type 3 — Commit Message Prompt:

"Write a conventional commit message for these changes: added a Redis caching layer to the /products endpoint which reduced average response time from 480ms to 35ms, and fixed a null pointer bug when product category is undefined."

Which doc type required the least editing? ☐ Docstring ☐ README ☒ **Commit Message**

Why? It's short and the AI already knew the exact conventional commit format (feat:, fix:, etc.), so there was barely anything for me to change.

Task 8 — Meeting Notes & Report Generation

15 min · Intermediate

Deliverable: 2 prompts written + audience suitability comparison noted.

Prompt 1 — Meeting Summary:

"Organize these raw meeting notes into a professional meeting summary with these sections: Attendees, Key Decisions, Action Items (with owner and deadline), and Blockers." [pasted the sprint planning notes]

Prompt 2 — Presentation Outline:

"Turn the same meeting notes into a 5-slide presentation outline, with a short title and 2-3 bullet points per slide."

Meeting summary is more useful for: ☐ Developers ☐ Management ☒ **All**

Presentation outline is more useful for: ☐ Developers ☐ Management ☒ **Client meetings**

Task 9 — Document Analysis & Data Extraction

25 min · Intermediate

Deliverable: 4 prompts written + quality ratings + JSON output from Prompt 3 pasted.

Prompt 1 — 5-Bullet Summary:

"Summarize this sprint report in exactly 5 bullet points, each under 15 words." [pasted the Sprint 14 report]

☒ Ready to use ☐ Needs minor edits ☐ Needs major revision

Prompt 2 — Action Items Table:

"Extract all action items from this report as a table with columns: Action, Owner, Deadline, Priority."

☒ Ready to use ☐ Needs minor edits ☐ Needs major revision

Prompt 3 — JSON Conversion:

"Convert the 'Performance Metrics' section of this report into a structured JSON object with a key for each metric."

☒ Ready to use ☐ Needs minor edits ☐ Needs major revision

Prompt 4 — Risk Analysis:

"Look at this sprint report, list the risks mentioned, and suggest one mitigation for each risk."

☐ Ready to use ☒ Needs minor edits ☐ Needs major revision

JSON output from Prompt 3:

```
{
  "sprint_velocity": {"actual": 42, "target": 45, "variance_percent": -6.7},
  "bugs_opened": 7,
  "bugs_resolved": 4,
  "bugs_open": 3,
  "code_review_turnaround_hours": {"actual": 18, "target": 24},
  "test_coverage_percent": {"actual": 74, "target": 80, "gap_percent": -6}
}
```

Task 10 — AI-Assisted Code Review, Debugging & Test Generation

25 min · Advanced

Deliverable: 3 CRTFC prompts + all results noted + reflection written.

```
def calculate_discount(cart_items, promo_code):
    """Apply discount to cart total."""
    PROMO_CODES = {'SAVE10': 0.10, 'SAVE20': 0.20, 'VIP50': 0.50}
    total = 0
    for item in cart_items:
        total += item['price'] * item['quantity']
    if promo_code in PROMO_CODES:
        discount = PROMO_CODES[promo_code]
        total = total - discount # BUG IS HERE
    if total < 0:
        total = 0
    return total
```

Sub-Task A — Code Review Prompt:

"Act as a senior Python developer. Review the function below for correctness, edge cases, security issues, and PEP8 compliance. List the issues found and suggest fixes."

What did AI find / produce?

It flagged that the discount is being subtracted as a flat number instead of applied as a percentage of the total — it should be "total * (1 - discount)", not "total - discount." It also noted there's no input validation for negative prices or quantities.

Sub-Task B — Bug Investigation Prompt:

"This function has a logic bug in how the discount gets applied. Identify the bug and give me a fixed version with an explanation."

What did AI find / produce?

Confirmed the bug: "total = total - discount" subtracts the raw decimal (like 0.20) instead of treating it as a percentage. Fixed version uses "total = total * (1 - discount)" so a 20% code actually takes 20% off instead of just \$0.20.

Sub-Task C — Test Generation Prompt:

"Generate pytest unit tests for the fixed version of this function. Cover: a normal cart with a valid promo code, an empty cart, a cart with no promo code, and a boundary case where the discount could make the total negative."

What did AI find / produce?

It generated 4 test functions, one for each case I listed, including a test that checks the total gets clamped at 0 instead of going negative.

Reflection — Which sub-task surprised you most and why?

The bug investigation part surprised me the most. I expected it to just say "looks fine" or give a generic answer, but it actually pinpointed the exact line causing the wrong math — that would've taken me a good while to notice on my own.

Task 11 — Build 3 Personal Reusable Prompt Templates

25 min · Intermediate

Deliverable: 3 complete reusable prompt templates with [PLACEHOLDERS].

General — Template 1: General Communication Template

Task this template is for: Weekly status update email to my manager

Context	[my role, team, and what happened this week]
Role	Act as a [project coordinator]
Task	[summarize this week's progress, blockers, and next week's plan in an email to my manager]
Format	[email, under 150 words, bullet points for blockers]
Constraints	[keep it casual but clear, no jargon]

Full prompt: "Act as a [project coordinator]. Write a weekly status update email to my manager covering: [this week's progress], [current blockers], and [next week's plan]. Keep it under 150 words, casual but clear, use bullet points for the blockers."

After testing: what 1 thing did you refine? Added a placeholder for a one-line "headline" at the top so my manager can skim it fast.

Dev — Template 2: Developer Workflow Template

Task this template is for: Writing a PR description for my code changes

Context	[what the PR changes and why]
Role	Act as a [software developer]
Task	[write a clear PR description summarizing the changes, why they were made, and how to test them]
Format	[markdown with sections: Summary, Changes, How to Test]
Constraints	[under 200 words, no fluff]

Full prompt: "Act as a [software developer]. Write a PR description for these changes: [describe the changes]. Include sections for Summary, Changes, and How to Test, in markdown, under 200 words, no fluff."

After testing: what 1 thing did you refine? Added a "Screenshots (if applicable)" section since a couple of my PRs are UI changes.

Custom — Template 3: My Choice Template

Task this template is for: Turning rough notes into a clean Slack update for my team

Context	[paste my rough notes]
Role	Act as a [team lead]
Task	[rewrite these notes into a clear, friendly Slack update]
Format	[short paragraphs, casual tone, use emoji sparingly]
Constraints	[keep it under 100 words]

Full prompt: "Act as a [team lead]. Rewrite these rough notes into a clear, friendly Slack update for the team: [paste notes]. Keep it under 100 words, casual tone, short paragraphs, use emoji sparingly."

After testing: what 1 thing did you refine? Told it to tone down the emojis — one per message max, it was a bit much at first.

Task 12 — Design Your Personal AI Developer Workflow

15 min · Advanced

Deliverable: Completed workflow table + 1 example prepared for group sharing.

#	Task Description	Type	AI Tool	Template Name	Est. Time Saved/Week
1	Writing my weekly status update ★	General	Claude	Template 1	30 min
2	Writing PR descriptions	Dev	ChatGPT	Template 2	20 min
3	Debugging error messages ★	Dev	Claude	Debug Helper	45 min
4	Drafting client-facing emails	General	ChatGPT	Template 1 (variant)	20 min
5	Generating unit tests ★	Dev	Claude	Test Gen Template	40 min
6	Summarizing meeting notes	General	ChatGPT	Meeting Summary	25 min
7	Writing commit messages	Dev	Claude	Commit Msg Template	10 min
8	Cleaning up rough Slack updates	General	Claude	Template 3	15 min

★ = my top 3 highest-impact tasks (circled): #1, #3, #5

My 60-Second Workflow Pitch:

The task I chose: Debugging error messages faster
AI tool I will use: Claude
The prompt template (summarize): I paste the error message plus a bit of the surrounding code, and ask it to explain what's causing it in plain language and suggest a fix, without rewriting the whole file.

Estimated time saved: about 45 minutes per week
Impact on my work: Less time stuck googling stack traces and stack overflow threads, more time actually building things.

Day 1 — Final Reflection

The most useful general AI use case I discovered today:

Turning messy meeting notes into a clean, organized summary — I always used to dread writing those up after a call.

The most useful developer AI use case I will start this week:

Using AI to review my code before I even open a pull request, so I catch small mistakes earlier instead of during review.

One thing I now understand about AI that I did not before:

How much the quality of the answer depends on how much context and detail you give it — a vague prompt really does just get you a vague answer back.

One question I still have about using AI in my development workflow:

I'm still not 100% sure what happens to the code or data I paste in — like whether it's ever stored or used to train the model later on.

My commitment — the first AI-assisted task I will do at work tomorrow:

Use AI to help write the PR description for the feature I'm finishing up today.